

✓ Valgrind e Simulação de Cache: **Cachegrind**

Este laboratório apresenta o uso da ferramenta cachegrind do ambiente Valgrind, [para maiores informações consulte aqui](#)

Importante:

- A primeira execução do Cachegrind irá fazer a instalação da ferramenta e pode demorar um pouco mais.
- Os laboratorios usam uma multiplicação de matrizes como exemplo. O tamanho da matriz cresce com $O(N^2)$ e o tempo de execução com $O(N^3)$.
- Os exemplos estão em C. Mas o Cachegrind trabalha sobre o executável e pode ser usado em qualquer binário.
- Fique a vontade para contribuir.

✓ Inicialização

Primeiro, configurar o laboratório.

```
!pip install git+https://github.com/lesc-ufv/cad4u >& /dev/null
!git clone https://github.com/lesc-ufv/cad4u >& /dev/null
%load_ext plugin
```

```
/content/cad4u/hdl/hdl.py:78: SyntaxWarning: invalid escape sequence '\%'
print("Ex. \%\%waveform <name_file>.vcd")
```

```
%%datacache
#include <stdio.h>
#include <stdlib.h>

long long moves = 0;

void hanoi(int n, char from, char to, char aux) {
    if (n == 1) { moves++; return; }
    hanoi(n - 1, from, aux, to);
    moves++;
    hanoi(n - 1, aux, to, from);
}

int main(int argc, char const *argv[]) {
    moves = 0;
    hanoi(20, 'A', 'C', 'B');
    return 0;
}
```

Installing. Please wait... done!

Data Cache

Size (kB)	16
Associative	8
Line (Bytes)	64

Start exe...

Parameters: 2048, 2, 32

LLi miss rate: 0.00%

D refs:	15,775,872	(8,423,442 rd + 7,352,430 wr)
D1 misses:	8,524	(6,566 rd + 1,958 wr)
LLd misses:	1,841	(1,296 rd + 545 wr)
D1 miss rate:	0.1%	(0.1% + 0.0%)
LLd miss rate:	0.0%	(0.0% + 0.0%)

LL refs:	9,664	(7,706 rd + 1,958 wr)
LL misses:	2,971	(2,426 rd + 545 wr)
LL miss rate:	0.0%	(0.0% + 0.0%)

Parameters: 4096, 2, 32

LLi miss rate: 0.00%

D refs:	15,775,872	(8,423,442 rd + 7,352,430 wr)
D1 misses:	5,778	(4,433 rd + 1,345 wr)
LLd misses:	1,841	(1,296 rd + 545 wr)
D1 miss rate:	0.0%	(0.1% + 0.0%)
LLd miss rate:	0.0%	(0.0% + 0.0%)

LL refs:	6,918	(5,573 rd + 1,345 wr)
LL misses:	2,971	(2,426 rd + 545 wr)
LL miss rate:	0.0%	(0.0% + 0.0%)

Parameters: 8192, 4, 64

LLi miss rate: 0.00%

D refs:	15,775,872	(8,423,442 rd + 7,352,430 wr)
D1 misses:	2,758	(2,109 rd + 649 wr)
LLd misses:	1,841	(1,296 rd + 545 wr)
D1 miss rate:	0.0%	(0.0% + 0.0%)
LLd miss rate:	0.0%	(0.0% + 0.0%)

LL refs:	3,898	(3,249 rd + 649 wr)
LL misses:	2,971	(2,426 rd + 545 wr)
LL miss rate:	0.0%	(0.0% + 0.0%)

Parameters: 16384, 8, 64

LLi miss rate: 0.00%

D refs:	15,775,872	(8,423,442 rd + 7,352,430 wr)
D1 misses:	2,265	(1,664 rd + 601 wr)
LLd misses:	1,841	(1,296 rd + 545 wr)
D1 miss rate:	0.0%	(0.0% + 0.0%)
LLd miss rate:	0.0%	(0.0% + 0.0%)

LL refs:	3,405	(2,804 rd + 601 wr)
LL misses:	2,971	(2,426 rd + 545 wr)
LL miss rate:	0.0%	(0.0% + 0.0%)

Specify all cache parameters

A extensão **%%cachegrind** é semelhante a linha de comando, importante que os tamanhos de cache devem ser potência de 2, a linha além de potência de 2 começa com 32 bytes. A ordem dos parametros é tamanho da cache, associatividade e tamanho da linha. Os flags para cache de dados, de instruções e de último nível são **D1**, **I1**, and **LL**, respectivamente.

```
%%cachegrind --D1=32768,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char const *argv[]) {

    int n = 100;
    int a[n][n], b[n][n], c[n][n];

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            a[i][j] = i + j;
            b[i][j] = i*2 + j;
        }
    }

    int temp;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            temp = 0;
            for (int k = 0; k < n; ++k) {
                temp += a[i][k] * b[k][j];
            }
            c[i][j] = temp;
        }
    }
    return 0;
}
```

```
LLi miss rate:      0.01%

D  refs:      12,318,285 (12,256,384 rd + 61,901 wr)
D1 misses:    133,115 ( 128,348 rd + 4,767 wr)
LLd misses:    9,077 ( 4,321 rd + 4,756 wr)
D1 miss rate:  1.1% ( 1.0% + 7.7% )
LLd miss rate: 0.1% ( 0.0% + 7.7% )

LL refs:      134,753 ( 129,986 rd + 4,767 wr)
LL misses:    10,688 ( 5,932 rd + 4,756 wr)
LL miss rate:  0.0% ( 0.0% + 7.7% )
```

✓ Atenção aos resultados

Valgrind quando simula a cache ele também simula a inicialização do sistema. Portanto, quando for utilizar o valgrind esteja ciente que se o seu código for muito simples será mascarado pela inicialização do sistema.

```
%%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
```

```
int main(int argc, char const *argv[]) {
    /// empty code
}
```

```
LLi miss rate:      1.02%

D  refs:      46,650 (35,077 rd + 11,573 wr)
D1 misses:    13,867 (11,365 rd + 2,502 wr)
LLd misses:    3,240 ( 2,216 rd + 1,024 wr)
D1 miss rate:  29.7% ( 32.4% + 21.6% )
LLd miss rate:  6.9% ( 6.3% + 8.8% )

LL refs:      15,438 (12,936 rd + 2,502 wr)
LL misses:    4,788 ( 3,764 rd + 1,024 wr)
LL miss rate:  2.4% ( 2.0% + 8.8% )
```

✓ Exemplo de código mascarado

Abaixo é apresentado um código de **transposição de matrizes** sendo mascarado pela inicialização do sistema.

```
%%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void trans(int M, int N) {
    int i, j, tmp;
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            tmp = A[i][j];
            B[j][i] = tmp;
        }
}
```

```

    }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            A[i][j] = i + j;

    trans(n, n); // transposição de matrizes
    return 0;
}

```

```

LLi miss rate:      0.80%

D  refs:           65,413 (50,698 rd + 14,715 wr)
D1 misses:         15,154 (11,498 rd + 3,656 wr)
LLd misses:         3,502 ( 2,222 rd + 1,280 wr)
D1 miss rate:       23.2% ( 22.7% + 24.8% )
LLd miss rate:       5.4% (  4.4% +  8.7% )

LL refs:           16,733 (13,077 rd + 3,656 wr)
LL misses:          5,058 ( 3,778 rd + 1,280 wr)
LL miss rate:        1.9% (  1.5% +  8.7% )

```

Resultados **somente inicialização** X **transposição de matrizes**

Somente Inicialização:

- D refs: 1,203,059 (771,626 rd + 431,433 wr)
- D1 misses: 284,860 (219,936 rd + 64,924 wr)

Transposição de matrizes:

- D refs: 1,221,822 (787,247 rd + 434,575 wr)
- D1 misses: 286,147 (220,069 rd + 66,078 wr)

Note que a diferença é pequena, sendo para a cache dados L1:

- D refs: 1221822 - 1203059 = 18763
- D1 misses: 286147 - 284860 = 1287

✓ Solução: Mais trabalho para o algoritmo

Uma solução é fazer com que o seu código der mais trabalho para cache de dados, assim a inicialização não irá mascarar os resultados.

✓ Transposição simples

```

%%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void trans(int M, int N) {
    int i, j, tmp;
    for (i = 0; i < N; i++)
        for (j = 0; j < M; j++) {
            tmp = A[i][j];
            B[j][i] = tmp;
        }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            A[i][j] = i + j;

    for (int i; i < 2000; ++i)
        trans(n, n); // transposição de matrizes
    return 0;
}

```

```

LLi miss rate:      0.00%

```

```

D   refs:      22,987,949 (18,805,319 rd + 4,182,630 wr)
D1  misses:    2,320,001 ( 269,370 rd + 2,050,631 wr)
LLd misses:     3,502 (   2,222 rd +   1,280 wr)
D1  miss rate:  10.1% (   1.4% +  49.0% )
LLd miss rate:   0.0% (   0.0% +   0.0% )

LL refs:      2,321,581 ( 270,950 rd + 2,050,631 wr)
LL misses:      5,059 (   3,779 rd +   1,280 wr)
LL miss rate:    0.0% (   0.0% +   0.0% )

```

```

%%cachegrind --D1=1024,8,32 --I1=32768,2,32 --LL=65536,2,32 --file
#include <stdio.h>
#include <stdlib.h>

#define n 32
int A[n][n], B[n][n];

void transpose_32_32(int M, int N) {
    int BLOCK_SIZE, rowIndex, colIndex, blockedRowIndex, blockedColIndex, eBlockDiagI, iBlockDiagI;
    BLOCK_SIZE = 8;
    for (colIndex = 0; colIndex < M; colIndex += BLOCK_SIZE) {
        for (rowIndex = 0; rowIndex < N; rowIndex += BLOCK_SIZE) {
            for (blockedRowIndex = rowIndex; blockedRowIndex < rowIndex + BLOCK_SIZE; ++blockedRowIndex) {
                for (blockedColIndex = colIndex; blockedColIndex < colIndex + BLOCK_SIZE; ++blockedColIndex) {
                    if (blockedRowIndex != blockedColIndex)
                        B[blockedColIndex][blockedRowIndex] = A[blockedRowIndex][blockedColIndex];
                    else {
                        eBlockDiagI = A[blockedRowIndex][blockedColIndex];
                        iBlockDiagI = blockedRowIndex;
                    }
                }
                if (colIndex == rowIndex)
                    B[iBlockDiagI][iBlockDiagI] = eBlockDiagI;
            }
        }
    }
}

int main(int argc, char const *argv[]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            A[i][j] = i + j;
    for (int i; i < 2000; ++i)
        transpose_32_32(n, n); // transposição de matrizes 32x32
    return 0;
}

```

```

I1  misses:      3,581
LLi misses:      3,514
I1  miss rate:    0.01%
LLi miss rate:    0.01%

D   refs:      29,162,328 (26,245,989 rd + 2,916,339 wr)
D1  misses:    1,844,982 ( 480,575 rd + 1,364,407 wr)
LLd misses:     48,387 (   21,298 rd +   27,089 wr)
D1  miss rate:   6.3% (   1.8% +  46.8% )
LLd miss rate:   0.2% (   0.1% +   0.9% )

LL refs:      1,848,563 ( 484,156 rd + 1,364,407 wr)
LL misses:      51,901 (   24,812 rd +   27,089 wr)
LL miss rate:    0.1% (   0.0% +   0.9% )

```

Processando os dados: Transposição simples X Transposição 32x32

Note que a cache de dados teve maiores valores na transposição 32x32:

- Transposição simples: 24,144,358
- Transposição 32x32: 29,162,358

Contudo as falhas na transposição 32x32 foram menores:

- Transposição simples: 10.7%
- Transposição 32x32: 6.3%

Logo, quanto menos falha na cache L1 melhor.

✦ Variando o tamanho da Cache e visualizando falhas e taxa de falhas

A extensão **%%rangecachegrind** executa várias vezes com tamanhos de cache especificados pela lista **datacache=(4,8,16,32)**, em Kbytes. O usuário especifica a associatividade (**ways**) e o tamanho do linha (**line**), os gráficos são gerados de forma automática.

```
%%rangecachegrind datacache=(1,2,4,8,16); ways=2; line=32; bargraph=(misses)

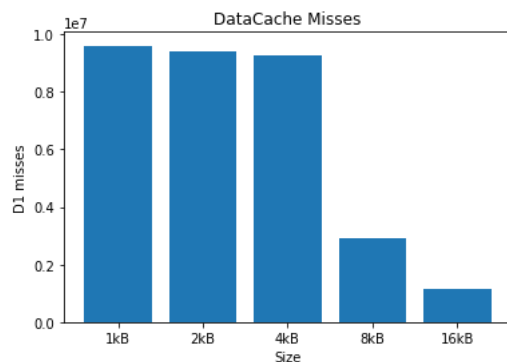
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char const *argv[]) {

    int n = 200;
    int a[n][n], b[n][n], c[n][n];

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            a[i][j] = i + j;
            b[i][j] = i*2 + j;
        }
    }

    int temp;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            temp = 0;
            for (int k = 0; k < n; ++k) {
                temp += a[i][k] * b[k][j];
            }
            c[i][j] = temp;
        }
    }
    return 0;
}
```



▼ Tarefa

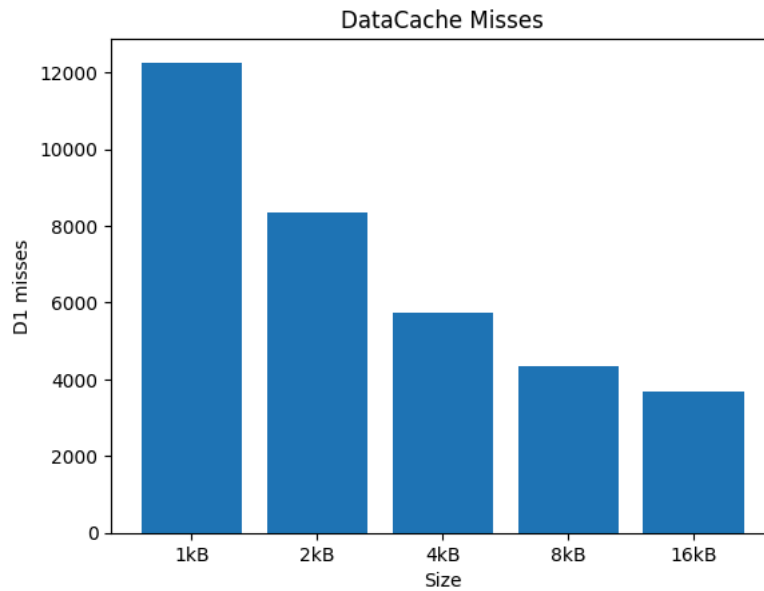
Variando os valores da cache de dados, ways e a lines (Utilize criatividade para mostrar)

```
%%rangecachegrind datacache=(1,2,4,8,16); ways=2; line=32; bargraph=(misses)
#include <stdio.h>
#include <stdlib.h>

long long moves = 0;

void hanoi(int n, char from, char to, char aux) {
    if (n == 1) { moves++; return; }
    hanoi(n - 1, from, aux, to);
    moves++;
    hanoi(n - 1, aux, to, from);
}

int main(int argc, char const *argv[]) {
    moves = 0;
    hanoi(20, 'A', 'C', 'B');
    return 0;
}
```



EXPLICAÇÃO DOS RESULTADOS

O gráfico acima mostra os D1 misses (falhas na cache de dados L1) do algoritmo de Torre de Hanoi recursivo com 20 discos, variando o tamanho da cache de 1 KB a 16 KB, com associatividade 2-way e linha de 32 bytes. Observa-se que à medida que o tamanho da cache aumenta, o número de misses diminui progressivamente. Com 1 KB, o número de misses é o mais alto (~12.000), pois a cache é pequena demais para armazenar os frames ativos da pilha de recursão. Conforme o tamanho cresce para 4 KB, 8 KB e 16 KB, os misses caem significativamente, pois mais frames da call stack conseguem permanecer na cache entre acessos consecutivos. Isso ilustra o princípio de localidade temporal: dados acessados recentemente tendem a ser acessados novamente em breve. No Hanoi recursivo, cada chamada acessa variáveis locais do frame atual e do frame pai — com cache maior, esses frames ficam disponíveis sem necessidade de busca na memória principal. Aumentar a associatividade (ways) reduz conflitos de mapeamento: com cache direta (1-way), diferentes endereços podem disputar a mesma linha de cache, causando evicções desnecessárias. Com 2-way ou mais, há mais flexibilidade de alocação, reduzindo esse problema. Aumentar o tamanho da linha (line) melhora a localidade espacial, trazendo mais dados vizinhos de uma só vez — útil para arrays, mas tem impacto menor no Hanoi, onde os acessos são majoritariamente à pilha de chamadas.